

General Framework of AI Agents

Hang Li: ACM Fellow, IEEE Fellow, CCF Distinguished Member

Abstract AI agents represent one of the most prominent frontiers in artificial intelligence. They include software agents that operate on PCs and mobile devices, as well as hardware agents (robots) that function in the physical world. Currently, research and development on AI agents are making significant progress in foundation models, training methods, system design, and practical applications. However, research on agent frameworks remains limited. This article, as a perspective paper, summarizes existing work and proposes a general framework for AI agent information processing. The key features of this framework are that agents are task-oriented; take text and multimodal data as inputs and outputs; use large language models for reasoning; are constructed through reinforcement learning; and leverage tools and memory. Representative agents and agent frameworks developed to date, as well as several agents recently developed at ByteDance Seed, all conform to this general framework. This article introduces these agents as examples. In addition, the article discusses the relationship between the proposed general agent framework and the information processing mechanisms of the human brain, analyzes the main characteristics of agent technologies, and proposes important directions for future research on AI agents.

Keywords AI Agents, Robots, Information Processing Framework, Multimodal Large Language Models

1 Introduction

The computer, also known as the ‘electronic brain,’ is the core of various intelligent tools and systems. The basic assumption here is that intelligence can be reduced to computation, and that through computational means we can realize intelligence.

Historically, computers have been successfully utilized in fields such as scientific computing, business systems, personal computing, internet, and smartphones, demonstrating a certain level of intelligence. Entering the era of large AI models, large language models and agents built on them have already exhibited capabilities approaching those of humans, raising the level of intelligence to greater heights. Looking ahead, AI systems are expected to reach or even surpass human intelligence in many aspects. However, the foundation of AI technologies remains computation.

Computers have also evolved from standalone operation to multi-machine interconnection, and to end-cloud collaboration. They also exist in various AI agents, including software agents and hardware agents

(robots). The computer in a robot acts as the ‘brain,’ planning and controlling the intelligent behavior of the entire body. Looking to the future, computers will become more diverse, ubiquitous, and omnipresent.

Software agents have gradually been adopted in many fields such as intelligent search, intelligent writing, intelligent programming, and intelligent customer service. Robots, especially those with manipulation capabilities, are expected to be put into practical use in fields such as manufacturing, logistics, and services in the coming years. All of this requires further in-depth research on AI agent technologies.

This article mainly discusses AI agents [1-6], especially the information processing frameworks of software agents and hardware agents (robots). We believe that AI agents should have a general information processing mechanism. In the mean time, AI agents should also have a unified computing architecture, much like the von Neumann architecture in traditional computers; for example, the recently proposed MCP is an attempt in this direction [7].

In fact, observing current AI agents reveals that their basic information processing frameworks have gradually taken shape. However, research and discussion on the general framework are not sufficient. This article attempts to summarize existing work, outline a general framework, and stimulate thinking and discussions. To the best of our knowledge, this is the first framework to encompass both software and hardware agents.

The main arguments of this article are as follows:

- Agents are designed to complete tasks. They take text and multimodal data as input and output. For hardware agents, they also produce physical action data. They rely on large language models for ‘thinking,’ are constructed through reinforcement learning, and utilize tools and memory when needed.
- The large language model (LLM) is the core of an agent, undertaking the ‘thinking’ function, and its capabilities determine the level of intelligence.
- There is a function level correspondence between the agent framework and the human brain’s information processing mechanism.
- Agents are capable of, and should be designed for, neural-symbolic processing.
- Future research directions for agents include: improving model architecture and training methods, scaling up data volume, developing autonomous and continual learning technologies, and enhancing security and controllability.
- If the reward function of agent reinforcement learning extends beyond the scope of task completion, it may pose significant risks, making it necessary to establish a red line for research and development.

The remainder of this article is organized as follows. Section 2 proposes a general framework for AI agents. Sections 3 and 4 review related work and introduce our work at ByteDance Seed, respectively. Section 5 compares AI agents with the human brain. Section 6 provides analysis and discussion, while Section 7 outlines important topics for future research. Section 8 concludes the article with final remarks.

2 General Framework of Agents

Main features

Artificial Intelligence agents, or AI agents, typically have the following characteristics:

1. An agent is a ‘machine that acts rationally,’ capable of interacting with the environment (including humans in it), aiming to complete tasks, and there are criteria for evaluating the quality of task completion.
2. The agent accepts text and multimodal data (including images, video, and audio) as input, and outputs text, multimodal data, and physical action data. However, only hardware agents (robots) execute physical action data.
3. The information processing mechanism from input to output in the agent is constructed with machine learning, particularly reinforcement learning. Reinforcement learning turns out to be the most effective means for implementing machine learning under such conditions.
4. The agent uses a multimodal large language model (MLLM) as its main component. The MLLM can handle multimodal inputs and produce multimodal outputs. These models are usually pre-trained and then further fine-tuned with reinforcement learning.

5. The large language model (LLM) in the agent serves as the agent’s brain for thinking and is vital to its behaviors. (Note that LLM is the main part of MLLM.) During the thinking process, it can generate chains of thought for reasoning. The agent’s thinking also includes planning, summarization, and reflection.

6. Agents utilize different tools, as well as memory, when needed. The use of tools and memory differentiates agents from LLMs themselves and, more importantly, enables a qualitative expansion of their capabilities. Tools may be built-in, such as a calculator, or externally accessible, such as a search engine, and there is no essential difference between the two from an information-processing perspective. Memory may include long-term memory and working memory (short-term memory).

7. Agents can have high or low autonomy according to the needs of the application, which refers to their ability to make decisions and take actions independently. Examples of the former include autonomous driving cars, while examples of the latter include intelligent customer services.

General framework

With the development of artificial intelligence technologies, the basic forms of information processing frameworks for AI agents have gradually taken shape. This article summarizes and organizes the frameworks of existing systems and proposes a new general framework (Fig. 1).

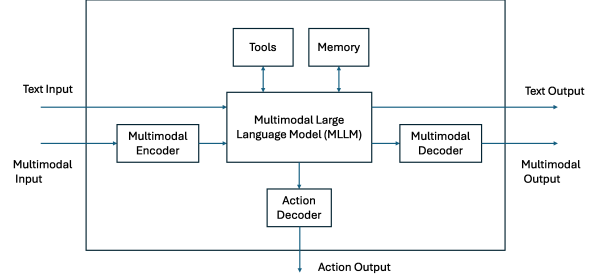


Fig. 1. General framework of AI agents.

The agent consists of an MLLM (containing LLM as main part), tools, memory (long-term memory and working memory), a multimodal encoder, a multimodal decoder, and an action decoder.

The agent can receive text input and provide text output, where the text can be natural language or formal language; the LLM generates text output from text input and may also generate text representing a reasoning process, i.e., a chain of thought. During the process, it may also use different tools and may read or write information and knowledge from or to the memory.

The agent can also receive multimodal inputs and provide multimodal outputs, including image, audio, and video; through the multimodal encoder, it generates multimodal intermediate representations, feeds them to the MLLM, generates new multimodal intermediate representations, and then generates multimodal outputs through the multimodal decoder; tools may be used during the process.

A hardware agent, i.e., a robot, can also receive text and multimodal inputs and provide physical action and multimodal outputs; physical actions can be the trajectories of the robots’ movement and manipulation; the action outputs are then executed by the robot’s hardware and control system.

Two types of model may both be needed in a hardware agent, MLLM itself, and multimodal-language-action model (MLAM) , which is MLLM augmented with action decoder. MLLM is for high-level task plan-

ning, reasoning, and interaction with the environment, while MLAM is for low-level action planning (i.e., generating trajectories to execute the plan).

Assume that the MLLM and multimodal encoder are basically trained through pre-training. The MLLM, multimodal encoder, multimodal decoder, and action decoder are further trained in post-training, including imitation learning and reinforcement learning. These basic modules are generally based on the Transformer architecture.

This framework has a two-layer structure: the bottom layer consisting of components such as the MLLM, encoder, decoder, tools, and memory, and the top layer that functions as the overall information processing mechanism. Furthermore, symbolic and neural representations are processed in these components.

3 Existing Work on Agents

For the agents in the field, such as AutoGPT, LangChain, ReAct, Reflexion, LATS, ToolFormer, Voyager, OS-Copilot, and Gemini Robotics 1.5, their information processing frameworks can all be considered special cases of the general framework shown in Fig. 1. This also applies to the four agents introduced in Section 4. While the inputs and outputs of these agents may vary, and they may utilize tools, memory, or both, their fundamental architectures and workflows are consistent. In the future, as agents become more general, so too will their underlying frameworks.

AutoGPT

AutoGPT is one of the earliest open-source framework for developing agents built on top of LLMs such as GPT-4 [8]. Given a task (e.g., generating a business plan for an online store), AutoGPT attempts to accomplish the goal through an iterative loop of planning, acting, and evaluation. It first decomposes the task

into smaller, actionable sub-tasks, then sequentially executes them using external tools such as web search, file I/O, and code execution. AutoGPT maintains a form of long-term memory, typically implemented via a vector database, to store intermediate results and context information.

LangChain

LangChain is a popular software development framework for building LLM-based agents (e.g., using GPT-4) [9]. It enables developers to construct systems and applications that accomplish complex, multi-step tasks, such as enterprise question answering, personal assistants integrating calendars and emails, or tools for database access and data analysis. LangChain provides abstractions including models, prompts, chains, tools, and memory. Chains allow multiple components to be composed into pipelines or workflows, while agents dynamically select and invoke tools such as search engines, databases, APIs, or code execution. Memory supports maintenance of conversational contexts and user-specific states.

ReAct

ReAct is a framework proposed to enhance LLMs by interleaving chain-of-thought reasoning with actions to interact with external tools [10]. This alternation continues until a task is solved. In question answering, ReAct reduces hallucination and error propagation by performing reasoning through interactions with tools such as Wikipedia API, and produces interpretable reasoning-action steps compared to prior baselines.

Reflexion

Reflexion is a framework that enables LLM-based agents to improve performance through self-reflection [11]. Instead of executing plans statically, Reflexion in-

troduces an iterative loop in which an actor attempts a task, an evaluator determines success or failure (often via external feedback), and upon failure a reflector generates insights in text about what went wrong and how to improve. These reflections are stored into memory and incorporated into future prompts, allowing the agent to avoid repeating past mistakes.

LATS

LATS is an agent framework that adapts Monte Carlo tree search for LLMs, integrating reasoning, acting, and reflection within a tree-based exploration process [12]. Starting from an initial task, the agent generates multiple candidate action steps, forming nodes in a search tree. These nodes are evaluated—often using LLM-based self-reflection to produce a scalar value or critique. A selection strategy expands high-value paths and prunes low-value ones. This process continues until a computational budget is reached, after which the highest-valued path is executed.

The results of tree search reasoning can be stored in the memory of the general framework, although it is not described in such a way in the original paper.

ToolFormer

Toolformer is an LLM trained to autonomously decide when and how to invoke external tools by emitting API calls directly in its generated text [13]. It has access to a fixed set of text-based APIs, including a calculator, search engine, question-answering system, translation system, and calendar. A key ingredient of Toolformer is its self-supervised training procedure. Starting from a text dataset, the model is prompted with few-shot examples to propose candidate API calls at various positions where tool use might be beneficial. These calls are executed against the actual tools, and only those that reduce the model’s next-token predic-

tion loss in the context are retained. The model is then fine-tuned on this filtered, tool-augmented dataset, enabling it to learn effective tool usage without manual annotations.

Voyager

Voyager is an agent developed for the game Minecraft, whose ultimate goal is to discover as many diverse elements in the environment as possible [14]. Voyager can autonomously and continuously explore its surroundings, acquire new skills, and make novel discoveries. It comprises three core components: an automatic curriculum for open-ended exploration, a library for storing and retrieving skills, and an iterative prompting mechanism for generating code. Voyager’s high-level planning and action generation are driven by its interactions with an LLM (GPT-4).

The workflow proceeds as follows. The curriculum proposes a new task based on the current state and successful and failed tasks. Relevant skills are then searched from the skill library. If no applicable skill is found, the prompting mechanism leverages GPT-4 to generate executable code as a skill. If the execution fails, GPT-4 is prompted again with the error feedback and the environmental observation to refine the code iteratively. Once the task is successfully completed, the code of a new skill is archived in the skill library. Overall, the agent comprises an LLM for planning (curriculum) and acting (prompting), and memory (library).

OS-Copilot

The OS-Copilot is an AI agent designed to act as a user’s assistant at the operating system level of a PC or smartphone [15]. Its primary role is to help the user to understand, control, and automate tasks across the entire system. It interacts with files, apps, settings, processes, and services within the OS. The interface is

natural language, i.e., commands in natural language.

The OS-Copilot framework consists of three components: a planner, a configurator, and an actor. The planner first decomposes a user’s request (task) into a sequence of sub-tasks and retrieves necessary tools and relevant information from the long-term and working memory. A plan is then created and passed to the configurator. The configurator combines information to create instructions on actions. Finally, the actor performs the actions, following an iterative executor-critic-refiner loop: the executor executes actions, the critic evaluates the result and provides repair suggestions, and the refiner rewrites and re-executes failed actions. Crucially, the entire framework supports continuous self-improvement, because successful actions and the critic’s feedback are stored in the long-term memory.

Gemini Robotics 1.5

Gemini Robotics 1.5 comprises two foundation models: an embodied-reasoning (ER) model and a vision-language-action (VLA) model [16]. Together, they enable robots to reason, plan, and execute tasks in real-world environments.

The ER model, apparently fine-tuned from an MLLM, serves as the system’s high-level cognitive component. It is responsible for understanding natural-language instructions, reasoning about physical spaces, and decomposing long-horizon tasks into elementary tasks. It conducts spatial and temporal reasoning through inferring object relationships and causal relation over time. It can also invoke external tools or function calls when necessary.

The VLA model, also apparently fine-tuned from an MLLM, serves as the system’s low-level perceptive and active component. It takes visual observations and language instructions as input and translates task plans

into robot movement and manipulation trajectories. The model performs implicit multimodal reasoning to align perception, language, and control. Additionally, the VLA model demonstrates strong cross-embodiment generalization, allowing skills learned on one robot to effectively transfer to others.

4 Our Work on Agents

This section introduces four AI agents, recently developed at ByteDance Seed.

AGILE

AGILE is an agent used for knowledge question answering (see Fig. 2) and is also one of the earliest agents trained using reinforcement learning [17]. The AGILE system includes an LLM, a search tool, and long-term memory. During the question answering process, the user submits a question to the LLM, and the LLM generates a response; its characteristic is that the beginning of the response is accompanied by an instruction. The system (specifically, the executor) performs the next operation based on the instruction, and the operations include: returning the response as an answer to the user, invoking the search tool to obtain an answer, consulting a human expert to obtain an answer, storing the answer in long-term memory, and conducting reflection and storing knowledge in long-term memory.

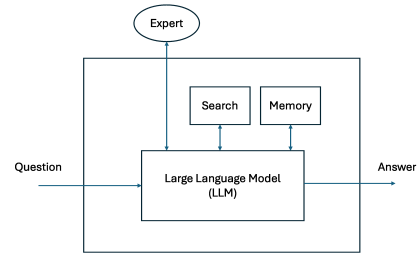


Fig. 2. Knowledge Q&A agent AGILE.

AGILE builds the entire system through reinforcement learning. The training of the LLM itself is the same as usual, where imitation learning (SFT) and

policy learning (PPO) are conducted on a pre-trained model.

Another characteristic of AGILE is that it can consult human experts. When an agent encounters a problem it cannot solve on its own, it consults an expert and receives guidance through dialogue. Frequent consultation with experts is not an ideal practice; in reinforcement learning, consulting an expert results in a lower reward, so agents are trained to consult experts as little as possible.

The performance of AGILE is tested on the question-answering task. After undergoing reinforcement learning with a certain amount of labeled data, the system can answer questions posed by humans more accurately and utilize resources more effectively, including long-term memory and experts. It is also observed that the agent possesses strong knowledge acquisition capabilities. For example, the role of expert consultation is not limited to the current task but also has an impact on future tasks, as knowledge acquired from experts may be retained in the long-term memory.

Delta Prover

Delta Prover is an agent capable of proving complex mathematical propositions. As illustrated in Fig. 3, it consists of an LLM and the Lean 4 prover [18]. Given a proposition expressed in the Lean 4 language, the agent can output a complete, machine-verifiable proof. Delta Prover employs two key algorithms during this process: Reflective Problem Decomposition (RPD) and Iterative Proof Repair (IPR). These algorithms continuously invoke the LLM and the Lean 4 prover.

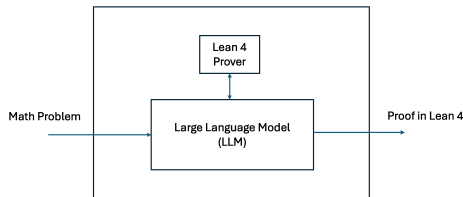


Fig. 3. Mathematical proof agent Delta Prover.

The traditional approach to mathematical proof involves fine-tuning of the model on manually written (annotated) Lean 4 proof data. This method has two significant drawbacks: 1. The proofs for complex propositions often consist of hundreds of lines of code, which require professional personnel to write, making the process extremely costly. 2. It is exceedingly difficult for an LLM to generate complex, logically rigorous code without errors, as its generation process is prone to hallucinations and error accumulation.

Delta Prover effectively addresses these issues. For a given problem, the RPD algorithm first directs the LLM to generate a proof sketch in Lean 4, thereby decomposing the original problem into multiple sub-problems. The Lean 4 prover then verifies the correctness of this sketch. If verification fails, the error information is returned to the LLM, which regenerates the proof sketch. This iterative process continues until verification is successful. Next, the IPR algorithm solves each sub-problem (if the original problem is simple enough, only IPR is needed). For each sub-problem, the IPR algorithm first has the LLM generate a complete proof. The Lean 4 prover then verifies this proof; if an error is found, the error information is returned to the LLM for repair. This process continues until verification is successful.

Since the LLM in Delta Prover only needs to generate relatively short proof outlines and proofs for simpler sub-problems, it effectively mitigates the issues associated with traditional methods. This system achieves a 96% success rate on the mathematical proof dataset miniF2F. This example demonstrates that using well-designed algorithms to orchestrate interactions between LLMs and specialized tools can significantly enhance the capabilities of AI agents.

Robix+GR-3

The models Robix [19] and GR-3 [20] together form a robotic agent capable of interaction and manipulation, as shown in Fig. 4. This work is concurrent with Gemini Robotics 1.5. Robix is an MLLM responsible for high-level interaction, understanding, and manipulation planning. GR-3 is a VLA (vision language action) model responsible for low-level manipulation control.¹ The agent takes video and audio as input and outputs speech and manipulation commands; these commands drive the robot hardware to complete various tasks. It also uses speech output to communicate with humans.

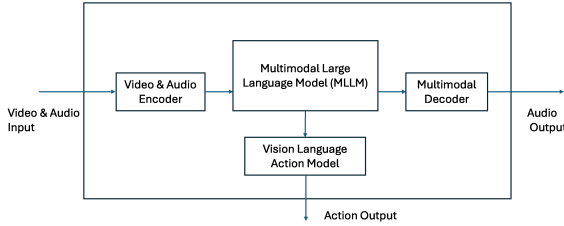


Fig. 4. Robotic agent Robix+GR-3.

The agent (robot) perceives and recognizes the surrounding environment (including people) through video and audio. It can initiate conversations with people, answer questions, follow instructions, generate commands for manipulation tasks (in natural language), and decompose complex tasks when necessary; it can understand and reason about the environment and can be interrupted by people during task execution. All these functions are performed by Robix. Then, based on the task instruction and the video information, the robot plans the future trajectory of its arm movement. All these functions are performed by GR-3. The entire process is streaming and occurs over time.

Robix is a unified MLLM that completes the above interactions, manipulation planning, and reasoning by

generating natural language instructions, responses, and chains of thought. GR-3 is also based on an MLLM but is designed to receive relatively simple instructions and does not accept audio input.

Directly using a general MLLM as the base model for robots is often inappropriate because robots need to understand the surrounding environment from their own perspective, especially the spatial and temporal relations. Therefore, it is necessary to continue training the general MLLM using data from the robot’s perspective to endow the model with the ability for ‘embodied understanding and reasoning.’

The training of Robix involves continual training, SFT, and reinforcement learning. Continual training aims to enhance the model’s embodied understanding and reasoning abilities; SFT and reinforcement learning are used to strengthen the model’s dialogue, manipulation planning, understanding, and reasoning capabilities. The training data includes synthetic samples covering all these tasks. GR-3’s training uses imitation learning, with (instruction, video, trajectory) samples as training data, while also incorporating a portion of (video, description) samples and human’s (instruction, video, trajectory) samples. Robix and GR-3 individually and in combination have achieved SOTA results on multiple test datasets.

M3-Agent

M3-Agent is a multimodal agent with long-term memory, which can be used in systems such as robots and personal intelligent assistants [21]. Future household robots may directly hand you a cup of coffee in the morning without asking “coffee or tea?” because through long-term interactions, they have gradually learned your habits. Such multimodal agents need to

¹In the future, it is very likely that the VLA model will be extended to a multimodal-language-action model, with inputs containing more modalities such as tactile data.

possess the following capabilities: to continuously perceive the environment through multimodal input, store experiences in long-term memory, gradually build world knowledge in long-term memory, and reason and even act based on world knowledge.

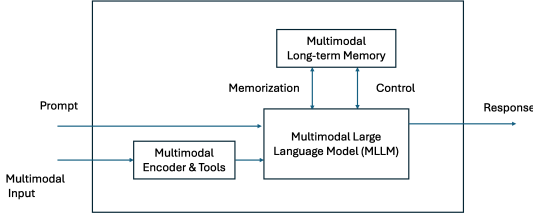


Fig. 5. multimodal agent with long-term memory M3-Agent.

As shown in Fig. 5, the robotic agent M3-Agent consists of an MLLM, multimodal encoders, tools, and multimodal long-term memory. The system operates through two parallel processes: memorization and control. During the memorization process, it continuously receives multimodal (video and audio) inputs from the environment to create and update world knowledge in its multimodal memory; during the control process, it reasons based on the world knowledge stored in memory according to external instructions and provides responses.

In memorization, M3-Agent, similar to humans, constructs and updates ‘episodic memory’ and ‘semantic memory.’ The former records observed events, while the latter records extracted knowledge fragments, which can be in the form of text, video, or audio. Its most prominent feature is that it constructs multimodal memory centered around entities to form an understanding of the environment, which constitutes world knowledge. For example, a person’s multimodal information (face, voice, etc.), a person’s attributes (name, gender, etc.), relationships between people, and relationships between people and objects are all represented in the graph in multimodality.

M3-Agent uses multimodal tools (facial recognition and speaker identification) to retrieve relevant knowl-

edge from long-term memory, generate episodic and semantic memories, and form new nodes and edges in the graph; all of the above operations are implemented by executing commands generated by the MLLM’s memorization policy model. This policy model is trained through imitation learning using data synthesized with semi-automatic methods.

In control, M3-Agent retrieves relevant knowledge from long-term memory according to instructions, conducts multiple rounds of reasoning and retrieval, and finally generates a response; these operations are also completed by executing commands generated by the MLLM’s control policy model. This policy model is trained using reinforcement learning and labeled data. The difference between M3-Agent and current RAG lies in its support for multi-round reasoning and retrieval, while RAG is generally single-round.

M3-Agent has achieved significantly higher accuracy compared to representative MLLMs in the online long-video question-answering task. This is because answering such questions requires processing cross-modal, cross-entity, and cross-event information in long videos, as well as understanding and reasoning about the scenes; M3-Agent has greater advantages here compared to traditional methods.

5 Comparison with Human Brain

Cognitive and computer scientist David Marr argued that computation can be defined on three levels: function, algorithm, and implementation. Artificial neural networks draw inspiration from the human brain at the implementation level, such as neurons and synapses. We contend that artificial intelligence, in general, should draw more inspiration from the human brain at the function level, to enhance its efficacy and efficiency. We point out that there is a correspondence between our proposed general framework and the mech-

anisms of the human brain at the function level.

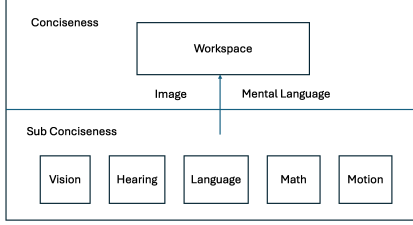


Fig. 6. Information processing mechanism of human brain.

The human brain consists of the cerebrum, the cerebellum, and the brainstem. The cerebrum is primarily responsible for thinking, including perception, cognition, and voluntary movement. The cerebellum mainly controls body movement and maintains balance. The cerebrum is divided into multiple functional regions, including those for vision, hearing, language (Broca’s area and Wernicke’s area), reasoning and planning (in the prefrontal cortex), mathematical operations (in the parietal lobe), and motion. The limbic system within the cerebrum regulates emotions (amygdala) and controls memory (hippocampus), among other functions.

Human thinking, i.e., the brain’s information processing, occurs mostly subconsciously, with multiple relatively independent subsystems processing information in parallel. The global workspace hypothesis in brain science posits that consciousness is the mechanism for achieving whole-brain information synchronization, and its information processing exhibits serial characteristics. This parallel-serial orchestrated mechanism of subconsciousness and consciousness enables the brain to maintain efficient processing while also responding effectively to the complex dynamics in the environment [22].

Embodied cognition theory posits that during human thinking, conscious processing produces ‘images,’² whereas the computational theory of mind posits that

it produces mental language (mentalese). Currently, there is no consensus on fully adopting either view [23]. This article draws on both theories and assumes that thinking can produce both mental language and images. Fig. 6 depicts the brain’s information processing mechanism.

We see similarities between the human brain and the general agent framework at the function level. First, both exhibit a two-layer structure: a lower layer of processing modules and an upper layer for orchestration or synchronization. Second, information is processed through these modules in both symbolic and neural representational forms. In other words, both human brain and AI agent employ neural symbolic processing, as further discussed later.

6 Analysis and Discussion

By combining a multimodal large language model (MLLM), tools, memory, and multimodal processing modules, the general agent obtains significant capability. This section explores key questions about the agent.

Software agent vs hardware agent

Software agents and hardware agents (robots) are essentially similar, but there are also differences. This is because they operate in different environments: software agents function in the digital world, while hardware agents work in the physical world. Although the environments they inhabit differ, both types of agents share the same information-processing framework. The main difference lies in their inputs and outputs within their respective environments.

Software agents usually take text and multimodal (visual and auditory) data as inputs, and typically produce symbolic or digital outputs, such as text, code,

²The term ‘image’ here refers to the visual, auditory, kinesthetic, and other representations that appear in a person’s mind. For example, when asked whether a gorilla has a nose, one must first recall what a gorilla looks like. The representation of a gorilla that comes to mind at that moment is the image of a gorilla.

or other structured representations. Hardware agents like robots, however, are supposed to process a larger variety of multimodal inputs. For example, they can incorporate tactile data. More importantly, their outputs are not limited to text and multimodal data, but include physical action data.

For instance, Voyager works in the Minecraft digital game environment. Its planning component (the automatic curriculum) takes a text representation of the agent’s current state and exploration progress as input. Its action component uses GPT-4 to generate executable code when a novel skill is required to accomplish the task.

In comparison, the Robix + GR3 (also Gemini Robotics 1.5) system works for a robot in a physical environment. The models take the user’s instruction and images of the environment as input. The high-level (MLLM) model generates responses and instructions for the low-level model (VLA). The VLA model can produce manipulation and movement trajectories.

Embodied cognition theory posits that human intelligence develops through interactions between the body and the environment [22]; this principle should therefore be extended to machine intelligence. Hardware agents, from this perspective, may benefit from richer input and action spaces, potentially enabling more general and versatile skills.

Large language model

The LLM, the main part of MLLM, serves as the ‘thinking brain’ of the agent and constitutes its core. The agent’s level of intelligence depends primarily on the capabilities of the LLM.

LLMs fine-tuned through reinforcement learning are themselves powerful agents. They accomplish tasks by interacting with humans through language generation. Interactions may be single-turn or multi-turn, during

which chain-of-thought reasoning may be conducted. The generated language can be natural language or formal language, such as code. The language generation of LLMs is context-based; in multi-turn interactions, this context refers to the interaction history up to that point, which is stored within the LLM’s context window or short-term memory.

LLMs also have limitations, however. They possess only short-term memory and cannot store or utilize information and knowledge indefinitely. Additionally, LLMs lack capabilities such as search, mathematical computation, and code execution. Moreover, they cannot directly process multimodal data.

To overcome these limitations, the agent incorporates long-term memory, tools, and multimodal processing modules, forming a two-layer structure. It is further retrained using reinforcement learning.

Reasoning

Reasoning or inference is a concept with multiple meanings and exists in a number of forms. In deep learning, inference usually refers to predicting unknown data using a trained model, and when the model is a generative model, it refers to generating new data. Logical reasoning in the field of mathematics (including propositional logic and first-order predicate logic) is the foundation of all mathematics. Bayesian inference and causal inference in machine learning each have a rigorous and complete mathematical system. Analogical reasoning refers to the process by which, for two similar things, the attributes, categories, and functions of one are extended to the other. It is generally held that in machine learning classification by neural networks, generation by language models, etc., are based on analogical reasoning.

What type of reasoning do humans use? Currently, science has no definitive conclusion. It appears to be

multi-faceted. When we prove mathematical theorems, we use logical reasoning in the derivation process. Sherlock Holmes determines who is the most likely criminal by using the collected evidence, and this process can be characterized by Bayesian inference. However, in daily thinking, humans more often use analogical reasoning, which plays an important role in human understanding, decision-making, and learning. For example, ‘time’ is an abstract concept, and we often understand it using a concrete concept, an arrowed straight line from left to right; this is essentially a metaphor.

The inference of large language models (LLMs) needs to be understood at three different levels:

The inference of pre-trained model is a process of auto-regressively predicting or generating the next token (next token prediction). In practice, it is a cyclic analogical inference based on the context token sequence. The objective during pre-training is to generate a token sequence that maximizes the likelihood or minimizes the cross-entropy loss, which is equivalent to lossless compression of the training data.

Fine-tuning through reinforcement learning (based on a verifier or RLHF) on the pre-trained model results in a model that acts as a policy function, where the state is represented by the token sequence of the current context and the action is represented by the next token. (SFT can be regarded as imitation learning). An inference path in reinforcement learning is the token sequence generated during the interaction between the LLM and the environment. The path with the maximum expected reward is the optimal token sequence. In training, a reward is ultimately obtained, while in inference, there is no reward. Therefore, at each step of inference, the model also makes a prediction for the next token.

The inference of an LLM can also be regarded as a search in the space of ‘statements.’ The subsequence

of tokens generated by the LLM forms a text segment, representing a complete semantic unit, which is referred to here as a statement. In fact, the inference of an LLM also takes place in the space composed of all possible statements. Statements can be based on natural language or formal language. They can represent logical reasoning or analogical reasoning. Chain of thought is also a type of statement. In this way, the inference (search) of an LLM can exhibit multiple types of reasoning abilities, especially for LLMs trained through reinforcement learning. The LLM in the agent is ultimately obtained through overall reinforcement learning training and inherits this reasoning ability.

Neural symbolic processing

Neural symbolic processing refers to a combination of symbolic processing and neural processing (deep learning). Agents are capable of, and should be designed for, neural symbolic processing, and this capability is a key differentiator between them and MLLMs (LLMs).

MLLMs (LLMs), encoders, and decoders perform neural processing. Although LLMs can participate in symbolic processing to some extent, they are not reliable for tasks requiring rigor. Tools, on the other hand, are designed to conduct symbolic processing. The knowledge in long-term memories is multimodal and should also include the outputs of symbolic processing.

For example, logical reasoning and mathematical computation are symbolic processing by nature and should be implemented as tools for agents rather than as capabilities of the LLMs. (Simple logical reasoning and basic calculations can, however, be handled by LLMs.) This is because LLM-based reasoning can only partially achieve rigorous logical reasoning and mathematical computation. In Delta Prover, verifying the

correctness of mathematical proofs is processed by the Lean 4 prover, which significantly enhances the accuracy of proving. This validates the rationality and necessity of implementing logical reasoning and mathematical computation as dedicated tools.

Moreover, world knowledge in long-term memory is inherently multimodal. A significant portion of this knowledge can be more naturally and reasonably represented in symbolic form. Scientific evidence suggests that human knowledge acquisition and memorization are centered around entities and concepts [24], which form a vast graph in the mind. Similarly, an agent’s memory can maintain such kind of a graph, which can be constructed from the output of an LLM through symbolic processing. The work of M3-Agent is a preliminary successful attempt in this regard.

An agent’s understanding of the environment, such as its comprehension of linguistic and visual inputs, should not be limited to superficial forms but must delve into semantics, that is, to achieve grounding. Accurate understanding of the environment enables the agent to fulfill tasks more effectively. The essence of grounding lies in associating new information with existing knowledge. Knowledge stored in long-term memory in neural-symbolic form can genuinely facilitate the grounding process.

7 Future Development

The development of agents and robots is still in its early stages, and numerous scientific and technological challenges remain to be explored and overcome. Beyond basic model architectures and training methods, the following important research topics also require extensive and in-depth investigation.

Expanding data scale

The lack of training data is likely the most significant

bottleneck currently hindering agent development. Collecting a sufficiently large amount of data in general or vertical domains for model training is an urgent issue that needs to be addressed and is the one that researchers are actively working on. For example, developing robots requires a sufficient number of robotic hardware systems to facilitate data acquisition.

One solution is to deploy reasonably competent agents in real-world scenarios to perform data acquisition, thereby obtaining a large amount of real-world data and establishing a closed loop for model training. Another approach is to use automated methods, including generative AI technologies, to synthesize data automatically for model training. These methods all require substantial experimentation and exploration in the future.

Autonomous and continual learning

Currently, agents are typically trained in advance and then deployed in specific scenarios. Many scientists have emphasized that future agents should be capable of autonomous and continual learning during utilization (e.g., [25]), meaning they learn through interactions with the environment.

Even within the current agent framework, more proactive learning is possible. For instance, agents could perform online reinforcement learning during operation. Through trajectory sampling, they would obtain environmental rewards, make trade-off between exploitation and exploration, and learn improved strategies. However, online reinforcement learning has not yet been widely adopted in practice, due in part to safety concerns.

Autonomous and continual learning means that an agent, through its interactions with the environment, continually constructs or updates its perception and understanding of the environment, continuously acquires

or refines its planning and control capabilities for task completion, and enhances its overall intelligence.

Security and controllability

The safety and controllability of agents will always remain among the most critical issues. Highly autonomous agents may pose significant risks, and minimizing the potential risks is of top priority.

If an agent is trained using reinforcement learning, a key consideration is how the reward function is defined. If the reward function reflects both task completion quality and alignment with human values, the associated risks, while present, remain relatively manageable.

However, if the reward function extends beyond task completion objectives, it could pose serious risks to humanity. It is essential to establish clear red lines in research and development. For example, if the reward function is based solely on an agent's ability to survive in an environment, the trained agent may develop selfish or even malicious behaviors. Agents learn through environmental interaction, just as with humans; the environment always contains information and knowledge that could lead them astray. This in itself is not inherently alarming. The crucial question is whether the agent can filter out harmful influences and learn 'good' skills and behaviors. Here, the design of the reward function plays a decisive role.

8 Concluding Remarks

This article discusses the general information processing framework of AI agents, arguing that the frameworks for both software agents and hardware agents (robots) have gradually evolved to a point where they can be unified and summarized. AI agents use advanced large AI model technologies, possess powerful

intelligent information processing capabilities, analogous to the intelligent processing mechanism of the human brain, and will become an important cornerstone of future artificial intelligence.

Undoubtedly, the information processing frameworks and computing architectures for future AI agents still require more research. This will drive the evolution of artificial intelligence, as it addresses the core issue of 'intelligence and computing' in computer science.

Acknowledgment

During the course of research at ByteDance Seed, I benefited greatly from discussions with numerous colleagues, which were instrumental in forming the ideas of this article. I would like to express my sincere gratitude to all of them. I am especially indebted to the anonymous reviewer, Andrew Estornell, Yuan Lin, Wei Li, Yichi Zhou, and Yang Wang for their invaluable feedback on the initial draft.

References

- [1] Xi Z, Chen W, Guo X, He W, Ding Y, Hong B, Zhang M, Wang J, Jin S, Zhou E, Zheng R. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 2025, 68(2), p.121101.
- [2] Liu B, Li X, Zhang J, Wang J, He T, Hong S, Liu H, Zhang S, Song K, Zhu K, Cheng Y. Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990*, 2025.
- [3] Masterman T, Besen S, Sawtell M, Chao A. The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584*, 2024.
- [4] Qu X, Damoah A, Sherwood J, Liu P, Jin C S, Chen L, Shen M, Aleisa N, Hou Z, Zhang C, Gao L. A comprehensive review of AI agents: Transforming possibilities in technology and beyond. *arXiv preprint arXiv:2508.11957*, 2025.

- [5] Weng L. LLM-powered autonomous agents. Lil’Log. 2023. <https://lilianweng.github.io/posts/2023-06-23-agent/>.
- [6] Zhang C, He S, Qian J, Li B, Li L, Qin S, Kang Y, Ma M, Liu G, Lin Q, Rajmohan S. Large language model-brained GUI agents: A survey. arXiv:2411.18279, 2024.
- [7] Anthropic. Model Context Protocol (MCP), 2023. <https://spec.modelcontextprotocol.io/>.
- [8] Significant Gravitas. Auto-GPT. GitHub Repository. 2023. <https://github.com/Significant-Gravitas/Auto-GPT>.
- [9] Chase H. LangChain. Python package, 2022, <https://github.com/langchain-ai/langchain>.
- [10] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K R, Cao Y. React: Synergizing reasoning and acting in language models. In Proc. of the 11th International Conference on Learning Representations, 2022.
- [11] Shinn N, Cassano F, Gopinath A, Narasimhan K, Yao S. Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems. 2023, 36:8634-52.
- [12] Zhou A, Yan K, Shlapentokh-Rothman M, Wang H, Wang Y X. Language agent tree search unifies reasoning, acting, and planning in language models. In Proc. of the 41st International Conference on Machine Learning. 2024, pp. 62138-62160.
- [13] Schick T, Dwivedi-Yu J, Dessi R, Raileanu R, Lomeli M, Hambro E, Zettlemoyer L, Cancedda N, Scialom T. Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems. 2023, 36:68539-51.
- [14] Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, Fan L, Anandkumar A. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291, 2023.
- [15] Wu Z, Han C, Ding Z, Weng Z, Liu Z, Yao S, Yu T, Kong L. OS-copilot: Towards generalist computer agents with self-improvement. arXiv:2402.07456, 2024.
- [16] Team G R, Abdolmaleki A, Abeyruwan S, Ainslie J, Alayrac JB, Arenas MG, Balakrishna A, Batchelor N, Bewley A, Bingham J, Bloesch M. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. arXiv:2510.03342, 2025.
- [17] Feng P, He Y, Huang G, Lin Y, Zhang H, Zhang Y, Li H. AGILE: A novel reinforcement learning framework of LLM agents. Advances in Neural Information Processing Systems, 2024, 37, pp.5244-5284.
- [18] Zhou Y, Zhao J, Zhang Y, Wang B, Wang S, Chen L, Wang J, Chen H, Jie A, Zhang X, Wang H. Solving formal math problems by decomposition and iterative reflection. arXiv:2507.15225, 2025.
- [19] Fang H, Zhang M, Dong H, Li W, Wang Z, Zhang Q, Tian X, Hu Y, Li H. Robix: A unified model for robot interaction, reasoning and planning. arXiv:2509.01106, 2025.
- [20] Cheang C, Chen S, Cui Z, Hu Y, Huang L, Kong T, Li H, Li Y, Liu Y, Ma X, Niu H. GR-3 technical report. arXiv:2507.15493, 2025.
- [21] Long L, He Y, Ye W, Pan Y, Lin Y, Li H, Zhao J, Li W. Seeing, listening, remembering, and reasoning: A multimodal agent with long-term memory. arXiv:2508.09736, 2025.
- [22] Dehaene S. Consciousness and the Brain, Deciphering How the Brain Codes Our Thoughts, Viking Press Publisher, 2014.
- [23] Li H. Intelligence and computing. Communications of the Computer Society. 2019, Vol. 15, in Chinese.
- [24] Patterson K, Nestor P J, Rogers T T. Where do you know what you know? The representation of semantic knowledge in the human brain. Nature Reviews Neuroscience. 2007, 8(12), pp.976-987.
- [25] Silver D, Sutton R S. Welcome to the era of experience. Google AI. 2025.